

NAME: P.Surendranath reddy

REG NO: 192472356

COURSE CODE: CSA0606

COURSE NAME: Design and Analysis of Algorithms

Selection Sort Using Brute Force Technique

Q. Apply the Brute Force technique using Selection Sort on the dataset [12, 5, 7, 5, 18, 9]. Clearly interpret the problem, explain how Selection Sort handles repeated minimum values, perform the sorting step-by-step, count comparisons and swaps, analyze time complexity, and verify the final output.

1. Objective

To apply the Brute Force technique using Selection Sort, sort the given dataset step-by-step, understand repeated minimum values, count comparisons and swaps, and analyze time and space complexity.

2. Problem Interpretation

The given array is [12, 5, 7, 5, 18, 9]. Selection Sort repeatedly searches the unsorted portion for the smallest element and places that minimum at the beginning of the unsorted portion.

3. Brute Force Approach

Selection Sort uses a straightforward exhaustive search for the minimum element in each pass. At every pass, it compares the current minimum with every remaining unsorted element.

4. Handling Repeated Minimum Values

The dataset contains two occurrences of the minimum value 5. In the first pass, one 5 is selected and placed at the first position. In the second pass, the remaining 5 is selected and placed at the second position. Thus, repeated minimum values are handled one at a time.

5. Selection Sort Procedure

1. Start with the first position as the current minimum.
2. Scan the remaining unsorted elements.
3. Find the smallest element.
4. Swap it with the first element of the unsorted portion if required.
5. Move the sorted boundary one position right.
6. Repeat until the array is sorted.

6. Step-by-Step Sorting Trace

Pass	Unsorted Portion	Minimum	Action	Array After Pass
1	[12, 5, 7, 5, 18, 9]	5	Swap 12 and 5	[5, 12, 7, 5, 18, 9]
2	[12, 7, 5, 18, 9]	5	Swap 12 and 5	[5, 5, 7, 12, 18, 9]
3	[7, 12, 18, 9]	7	Already in position; no swap	[5, 5, 7, 12, 18, 9]
4	[12, 18, 9]	9	Swap 12 and 9	[5, 5, 7, 9, 18, 12]
5	[18, 12]	12	Swap 18 and 12	[5, 5, 7, 9, 12, 18]

7. Comparison Count

For $n = 6$ elements:

$$(n - 1) + (n - 2) + (n - 3) + (n - 4) + (n - 5) \\ = 5 + 4 + 3 + 2 + 1 = \mathbf{15 \text{ comparisons.}}$$

8. Swap Count

Swaps occur in Pass 1, Pass 2, Pass 4, and Pass 5. Pass 3 requires no swap. Therefore, total swaps = **4**.

9. Intermediate Array Summary

Initial	[12, 5, 7, 5, 18, 9]
After Pass 1	[5, 12, 7, 5, 18, 9]
After Pass 2	[5, 5, 7, 12, 18, 9]
After Pass 3	[5, 5, 7, 12, 18, 9]
After Pass 4	[5, 5, 7, 9, 18, 12]
After Pass 5	[5, 5, 7, 9, 12, 18]

10. Time Complexity Analysis

Best Case: $O(n^2)$ — Selection Sort still scans the remaining unsorted elements even when the array is already sorted.

Average Case: $O(n^2)$ — The algorithm performs approximately $n^2/2$ comparisons regardless of the initial arrangement.

Worst Case: $O(n^2)$ — The algorithm performs $n(n-1)/2$ comparisons. Swaps are at most $O(n)$.

Space Complexity: $O(1)$ — The array is sorted in place using constant extra space.

11. Correctness Verification

The final output is **[5, 5, 7, 9, 12, 18]**. It is in non-decreasing order, and every original element appears exactly once. Both repeated 5s are retained and placed at the beginning. Hence, the result is correct.

12. Interpretation of Algorithm Behavior on Duplicates

Selection Sort does not remove duplicates. Each occurrence is treated as a separate element. After the first 5 is placed, the second 5 remains in the unsorted portion and is selected during the next pass. Duplicate values do not reduce the normal comparison count.

13. Performance Summary

Parameter	Result
Input Size	6 elements
Comparisons	15
Swaps	4
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

Parameter	Result
Final Output	[5, 5, 7, 9, 12, 18]

14. Advantages

- Simple and easy to implement.
- Works in-place with $O(1)$ extra space.
- Predictable comparison count.
- Correctly handles repeated values.

15. Limitations

- $O(n^2)$ time complexity in all cases.
- Performs many comparisons even when the input is already sorted.
- Generally not stable because swapping can change the relative order of equal-valued records.

16. Conclusion

Selection Sort successfully sorts [12, 5, 7, 5, 18, 9] using the Brute Force technique. It makes 15 comparisons and 4 swaps and produces [5, 5, 7, 9, 12, 18]. The repeated minimum value 5 is handled one occurrence at a time. The algorithm has $O(n^2)$ time complexity in the best, average, and worst cases and $O(1)$ auxiliary space.